1. Gram-Schmidt Orthogonalization: To find orthogonal basis vectors for the given set of vectors and plot the orthonormal vectors.

CODE:

```
% Given set of vectors as columns
V = [1 0 0; 1 1 1; 0 0 1]';
% Number of vectors
num_vectors = size(V, 2);
% Initialize orthogonal and orthonormal matrices
U = zeros(size(V));
E = zeros(size(V));
% Gram-Schmidt Process
U(:,1) = V(:,1);  % First orthogonal vector is the first input vector
E(:,1) = U(:,1) / norm(U(:,1));  % First orthonormal vector
for i = 2:num_vectors
  U(:,i) = V(:,i);
  for j = 1:i-1
    U(:,i) = U(:,i) - (dot(V(:,i), E(:,j)) * E(:,j)); % Subtract projection onto previous orthonormal vectors
  end
  E(:,i) = U(:,i) / norm(U(:,i));  % Normalize to get orthonormal vector
end
% Display the orthonormal vectors
disp('Orthonormal basis vectors:');
disp(E);
% Plot the original and orthonormal vectors
figure;
hold on;
grid on;
axis equal;
```

```
% Plot original vectors
quiver3(0, 0, 0, V(1,1), V(2,1), V(3,1), 'r', 'LineWidth', 2);
quiver3(0, 0, 0, V(1,2), V(2,2), V(3,2), 'g', 'LineWidth', 2);
quiver3(0, 0, 0, V(1,3), V(2,3), V(3,3), 'b', 'LineWidth', 2);
% Plot orthonormal vectors
quiver3(0, 0, 0, E(1,1), E(2,1), E(3,1), 'r--', 'LineWidth', 2);
quiver3(0, 0, 0, E(1,2), E(2,2), E(3,2), 'g--', 'LineWidth', 2);
quiver3(0, 0, 0, E(1,3), E(2,3), E(3,3), 'b--', 'LineWidth', 2);
xlabel('X');
ylabel('Y');
zlabel('Z');
legend('Original V1', 'Original V2', 'Original V3', 'Orthonormal E1', 'Orthonormal E2',
'Orthonormal E3');
title('Original and Orthonormal Vectors');
hold off;
```

3 Perform the QPSK Modulation and demodulation. Display the signal and its constellation

Code:

```
clc;
clear all;
close all;

% Input bit sequence
bit_seq = [1 1 0 0 0 0 1 1];
N = length(bit_seq);
fc = 1; % Carrier frequency
t = 0:0.001:2; % Time vector
b = []; % Input bit sequence as a waveform
qpsk1 = []; % QPSK signal
bec = []; % Even bit cosine waveform
bes = []; % Odd bit sine waveform
b_o = []; % Odd bit stream
b_e = []; % Even bit stream
```

```
bit_e = []; % Even bit stream (for plotting)
bit_o = []; % Odd bit stream (for plotting)

% Creating input waveform from bit sequence
for i = 1:N
  bx = bit_seq(i) * ones(1, 1000); % Generate pulse for each bit
  b = [b, bx];
end

% Modifying bits for QPSK mapping: 0 -> -1
for i = 1:N
  if bit_seq(i) == 0
    bit_seq(i) = -1;
  end
  if mod(i, 2) == 0
    e_bit = bit_seq(i);
    b_e = [b_e, e_bit];
  else
    o_bit = bit_seq(i);
    b_o = [b_o, o_bit];
  end
end

% Generate QPSK modulated signal
for i = 1:N/2
  % Even bits modulated on cosine wave
  be_c = (b_e(i) * cos(2*pi*fc*t));
  % Odd bits modulated on sine wave
  bo_s = (b_o(i) * sin(2*pi*fc*t));
  q = be_c + bo_s; % Combine both to form QPSK signal

  % Collect even and odd bit streams for plotting
  even = b_e(i) * ones(1, 2000);
  odd = b_o(i) * ones(1, 2000);
  bit_e = [bit_e, even];
  bit_o = [bit_o, odd];
  qpsk1 = [qpsk1, q];
  bec = [bec, be_c];
  bes = [bes, bo_s];
end

% Plotting the QPSK signals and constellations

figure('name', 'QPSK Modulation');
subplot(5, 1, 1);
plot(b, 'o');
grid on;
```

```
axis([0 N*1000 0 1]);
title('Binary Input Sequence');

subplot(5, 1, 2);
plot(bes);
hold on;
plot(bit_o, 'rs:');
grid on;
axis([0 N*1000 -1 1]);
title('Odd Bits (Sine Component)');

subplot(5, 1, 3);
plot(bec);
hold on;
plot(bit_e, 'rs:');
grid on;
axis([0 N*1000 -1 1]);
title('Even Bits (Cosine Component)');

subplot(5, 1, 4);
plot(qpsk1);
axis([0 N*1000 -1.5 1.5]);
title('QPSK Modulated Signal');

% QPSK Constellation Plot
subplot(5, 1, 5);
% QPSK constellation points
constellation = [1 + 1j, -1 + 1j, -1 - 1j, 1 - 1j];
plot(real(constellation), imag(constellation), 'bo', 'MarkerSize', 8, 'LineWidth', 2);
grid on;
axis([-2 2 -2 2]);
title('QPSK Constellation');
xlabel('In-phase (I)');
ylabel('Quadrature (Q)');
```

Theory:

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique used to transmit data efficiently over communication channels. It is a type of phase modulation where data is represented by four distinct phase shifts of a carrier signal. QPSK uses two bits per symbol, meaning each symbol carries two bits of information.

In QPSK, the carrier signal is modulated by varying its phase. The four possible phase shifts (0°, 90°, 180°, and 270°) correspond to different pairs of bits:

- $00 \rightarrow 0°$
- $01 \rightarrow 90°$
- $10 \rightarrow 180°$

Exp no 2. Simulation of binary baseband signals using a rectangular pulse and estimate the BER for AWGN channel using matched filter receiver.

CODE:


```
% Simplified Parameters
N = 1e4;
SNR_dB = 0:5:20;
pulse_width = 1;
% Number of bits
% SNR values in dB
% Pulse width for rectangular pulse
% Generate random binary data
data = randi([0 1], N, 1);
```

```
% Define the rectangular pulse
t = 0:0.01:pulse_width;
rect_pulse = ones(size(t));
% Initialize BER vector
BER = zeros(length(SNR_dB), 1);
for snr_idx = 1:length(SNR_dB)
% Modulate binary data
tx_signal = [];
for i = 1:N
if data(i) == 1
tx_signal = [tx_signal; rect_pulse'];
else
tx_signal = [tx_signal; zeros(size(rect_pulse'))];
end
end
% Add AWGN
SNR = 10^(SNR_dB(snr_idx) / 10);
noise_power = 1 / (2 * SNR);
noise = sqrt(noise_power) * randn(length(tx_signal), 1);
rx_signal = tx_signal + noise;
% Matched Filter
matched_filter = rect_pulse;
filtered_signal = conv(rx_signal, matched_filter, 'same');
% Sample the output of the matched filter
sample_interval = round(length(filtered_signal) / N);
sampled_signal = filtered_signal(1:sample_interval:end);
% Decision (Threshold = 0.5)
estimated_bits = sampled_signal > 0.5;
```

```
% Compute BER
num_errors = sum(estimated_bits ~= data);
BER(snr_idx) = num_errors / N;
end
% Plot BER vs. SNR
figure;
semilogy(SNR_dB, BER, 'b-o');
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('BER vs. SNR for Rectangular Pulse Modulated Binary Data');
```

Theory :